

MTRX3760 LAB 1 REPORT

A1 - Robot

Function-Based

To be completed with the function-based version.

Object-Based

Program Output

```
Left motor 0.8, Right motor 0.2
Battery charge: 90
Left motor 0.55, Right motor 0.45
Battery charge: 80
Left motor 0.05, Right motor 0.45
Battery charge: 70
Left motor 0, Right motor 0.5
Battery charge: 60
```

Questions

When you added the battery, what changed in `main()` in each version? In the function-based version, `main()` had to manage the battery state and pass or update it each cycle. In the object-based version, `main()` did not need to change because the battery was added inside `CRobot`.

In the function-based version, what does `main()` have to know about the robot's parts? What does it know in the object-based version? In the function-based version, `main()` has to know about individual parts such as the sensor, motors, and battery. In the object-based version, `main()` only needs to know about the `CRobot` object and call its `Update()` and `Report()` functions.

What is the name of the object-oriented principle that produced this saving? Encapsulation. The robot's internal parts and their behaviour are kept inside `CRobot`, so `main()` does not need to manage them directly.

A2 - Oven

Private

Public

Program Output

```
Reflow oven is at 25 C
```

```
Curing oven is at 28 C
```

Questions

Which version required more changes? Name the functions or lines you had to edit in each. The public-data version required changes in several places inside `main()`: the starting temperatures changed from 20 to 200, warming changed from +1 to +10, the overheating limit changed from 250 to 2500, and the reported temperature had to be divided by 10 before printing.

(Check with private version, then answer)

What is the object-oriented principle that kept the change contained? Encapsulation. The temperature is private in the encapsulated version, so the details of how it is stored can be changed inside `COven` without other code needing to know.

Did `main()` need to change in the public version? In the private version? Explain. Yes, `main()` had to change in the public version because it directly accesses and modifies `mTemperatureC`. In the private version, `main()` should not need to change because it uses functions such as `WarmUp()`, `IsOverheating()`, and `Report()` instead of depending on how the temperature is stored internally.

A3 - Clock

Inherit

Standalone

Program Output

```
Bedroom 07:00  
Kitchen 07:00
```

Questions

In the standalone version, which parts of the clock's behaviour did you have to repeat? The standalone `CAAlarmClock` had to repeat the normal clock's time data and behaviour, including storing the current time, advancing it with `Tick()`, and formatting the time in `Report()`.

In the inheritance version, what did you add, and what did you get without writing it? *To be completed.*

What is the name of the object-oriented principle that let the second version reuse the clock's behaviour? Inheritance. It allows `CAAlarmClock` to reuse the existing behaviour of `CClock` while adding its own alarm-specific features.

A4 - Power Budget

Power_budget1

Power_budget2

Program Output

```
DriveMotor: 60 W
StatusLed: 0 W
Heater: 60 W
Total: 120 W
```

Questions

What did you have to change in each version to add the heater? In `power_budget2.cpp`, a new `CHeater` class was added as a subclass of `CDevice`. It was given a heat setting, a `SetHeat()` function, and its own `PowerDraw()` implementation. A heater object was then created in `main()` and added to the `devices` array.

Which version was easier to extend, and what specifically made it easier? `power_budget2.cpp` was easier to extend because all devices are stored as `CDevice*` objects and processed using the same loop. Once `CHeater` implemented its own `PowerDraw()` function, it could simply be added to the device array without needing a separate block of reporting and total-calculation code.

What is the name of the object-oriented principle behind that difference? Polymorphism. The motor, LED, and heater can all be treated as `CDevice` objects while each class provides its own implementation of `PowerDraw()`.